

WHAT'S MINE IS MINE

Personal, automatic, lifecycle-aware file synchronization

Concept + Architecture + 24-hour MVP plan

One-line thesis

A Linux synchronization utility that stays beneath normal computer usage: permanently replicating the user-selected files that belong to their working environment, while also providing temporary cross-machine replication for short-lived data that should disappear automatically without requiring the user to remember cleanup.

1. Problem

People increasingly use more than one computer as parts of the same personal working environment. A desktop may be the primary machine at home while a laptop is used at college, work, or while travelling. The friction is not that synchronization is impossible; it is that remembering what needs to be synchronized is itself another task.

The intended workflow is therefore deliberately invisible: configure the utility once, specify what data matters, start it, and then continue using the computer normally. The user should eventually develop the expectation that designated data is simply available wherever they work.

2. Concrete use cases

User / workflow	Example
Student / developer	Project PDFs, study material, notes, and small working artifacts follow between a PC and laptop.
Linux power user	Selected configuration such as .bashrc, .gitconfig, and chosen ~/.config trees stay aligned without syncing machine-specific configuration.
Photographer / creator	Selected images or exports can become available on another workstation without manually copying them each time.
Everyday user	Documents, reference material, wallpapers, and other chosen files remain available across machines.
Temporary transfer	A save file, archive, image, or document is needed on another computer for a short period and should not become permanent clutter.

3. Product model

The local filesystem remains the user's filesystem. The utility does not require a special sync directory or force data to be reorganized. The user defines roots and selection rules; the utility watches those locations and represents the selected data in a cloud-backed synchronization state.

Example rules

```
~/Projects          -> **/*.pdf
~/Documents/Study   -> **/*.pdf
~/Pictures/Wallpapers -> *.{png,jpg,webp}
~/.bashrc           -> exact file
~/.gitconfig        -> exact file
```

Relative paths under each configured root are preserved, so a PDF inside Projects/Hevel remains associated with Projects/Hevel rather than being flattened into one remote pile.

Design principle
The utility conforms to the user's existing organization instead of making the user reorganize their data around the synchronization system.

4. Temporary synchronization: TMP + Recall

The permanent synchronization path is not enough for files that are needed only briefly. TMP is an explicit lifecycle-aware mode for short-lived cross-machine data. The user places an item into temporary synchronization once; after that, the system takes responsibility for propagation and cleanup.

```
local machine -> TMP -> cloud -> other machine

Upload completes at 14:01
Active lifetime:      until 14:01 next day
Recall window:        next 24 hours
Permanent deletion:  after recall window
```

The 24-hour active lifetime begins when the item is successfully committed to the cloud, not when the user first requests the upload. This makes the cloud the authoritative clock for temporary lifecycle state.

5. Recall history

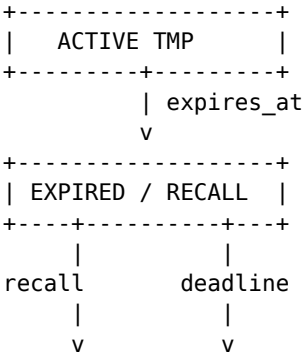
Once an item expires from the active temporary store, the cloud retains a short-lived recovery copy during the Recall window. Recall is not another permanent sync state; it is a bounded recovery tier for human forgetfulness.

```
wmine tmp list

ID      FILE                                STATE      TIME LEFT
a812    Deadzone/save.sav                     RECALLABLE 3h 12m
b941    college/assignment.pdf                ACTIVE      9h 04m
c230    wallpaper.webp                        RECALLABLE 23h 51m
```

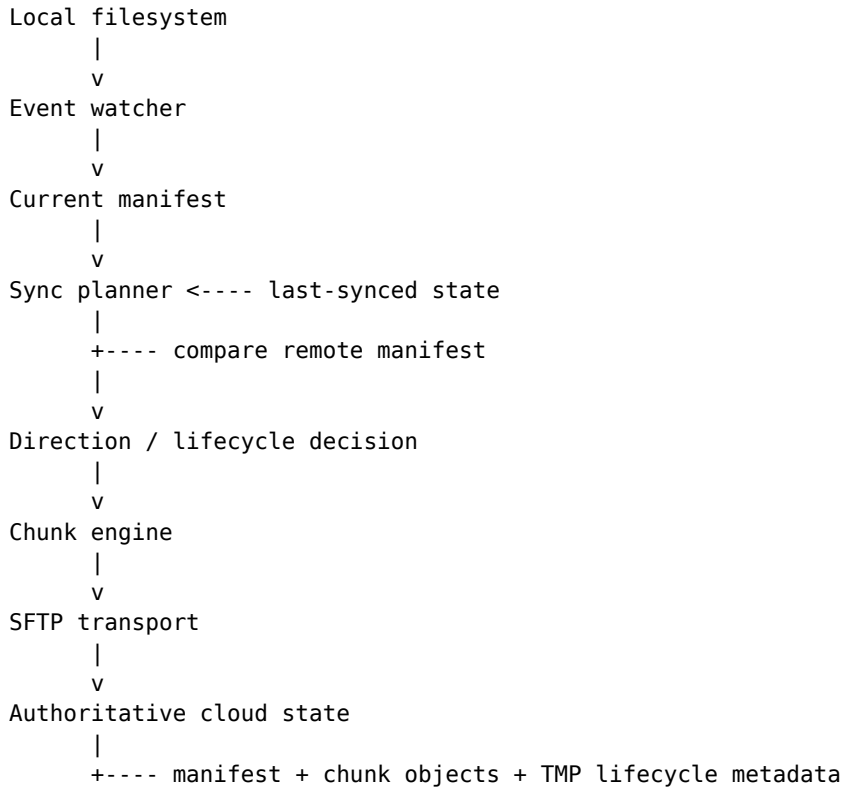
If recalled, the item returns to active TMP with a fresh 24-hour lifetime. When the Recall window ends, the retained cloud copy is permanently deleted. The history keeps metadata about recently expired items so the user can remember what disappeared even after the data is gone.

6. Lifecycle state machine



Permanent sync data follows a separate persistent path and does not enter this lifecycle.

7. Architecture



8. Core components

Component	Responsibility
Configuration	Defines sync roots, include patterns, local-to-remote mappings, and TMP policy.
Watcher	Records filesystem changes as they happen. Synchronization is rate-limited; changes are not silently ignored.
Manifest	Tracks paths, metadata, timestamps, chunk hashes, and temporary lifecycle metadata.
Sync planner	Compares current local state, last-synced state, and authoritative remote state before doing transfers.
Chunk engine	Uses fixed-size chunks for the MVP. Changed chunk hashes determine which portions need transfer.
Transport	SFTP is the first backend: simple, available, and independent of a provider-specific cloud API.
Lifecycle manager	Maintains active TMP expiry, Recall retention, and final deletion based on cloud-authoritative timestamps.
History	Keeps metadata about recently expired TMP items without retaining the file indefinitely.

9. Deliberate engineering decisions

Five-minute synchronization cooldown

The utility is intentionally quiet. Filesystem changes are observed immediately, but a sync attempt is limited to at most once per configured interval. This matches personal, asynchronous use better than aggressive real-time traffic.

Cloud is the source of truth

The machines are replicas rather than peers. This is especially important for TMP expiry: an offline machine must not resurrect a file that the cloud has already expired.

Timestamp-based direction

For the MVP, timestamp priority determines whether local or remote state should win. Simultaneous multi-writer conflict resolution is outside the intended single-user workflow.

Fixed chunks with periodic rebuilding

Fixed-size chunks keep normal synchronization simple and predictable. Periodic full rechunking and rehashing can recover efficiency after repeated insertions or deletions cause poor chunk alignment.

Explicit selection rules

The user chooses roots and file patterns. The utility does not infer what data is important and does not reorganize the user's filesystem.

One transport first

SFTP is sufficient to demonstrate the architecture. Provider-specific integrations are intentionally deferred so the MVP proves synchronization semantics rather than OAuth/API integration.

10. State model and consistency rules

Current local state = what exists on disk now
Last-synced state = what the client believes was previously synchronized
Remote state = authoritative cloud manifest

Local newer -> upload
Remote newer -> download
Same state -> no action

Temporary: now < expires_at -> active
Temporary: expires_at <= now < recall_deadline -> recallable
Temporary: now >= recall_deadline -> permanently deleted

A safe transfer follows plan -> transfer -> verify -> commit. The manifest is only advanced after the corresponding data is verified, so an interrupted transfer can resume without treating incomplete state as authoritative.

11. 24-hour MVP boundary

The assessment build should prove the central workflow rather than attempt production-grade synchronization. The smallest convincing implementation is:

- One Linux CLI executable.
- Simple configuration for sync roots and include patterns.
- Filesystem watcher with a five-minute sync cooldown.

- Local and remote manifests.
- Timestamp-based upload/download decisions.
- Fixed-size chunk hashing and changed-chunk transfer.
- SFTP upload/download transport.
- Permanent synchronization of selected files.
- TMP with 24-hour active lifetime.
- 24-hour Recall window with cloud-side retention.
- Recent-expiry history.
- Basic integrity verification and resumable transfer behaviour.

Explicitly out of scope for the assessment MVP

GUI, multiple cloud providers, collaborative conflict resolution, content-defined chunking, machine-learning classification, full-system backup, encryption protocol design, and polished packaging/distribution.

12. Demonstration plan

Demo	What it proves
Permanent path	Configure Projects/**/*.*.pdf and demonstrate an edited PDF arriving automatically on the second machine in its corresponding subdirectory.
Temporary path	Place a game save or other short-lived file into TMP, show it arriving on the second machine, then show its expiry state.
Recall	Let an item enter Recall, list the remaining retention time, recall it, and demonstrate the renewed 24-hour active lifetime.
Offline expiry	Keep one client offline through expiry, reconnect it, and show that cloud-authoritative state prevents stale data from being resurrected.
Failure recovery	Interrupt a transfer, restart synchronization, and show that verified pieces are reused rather than forcing a complete restart.

13. Known limitations and future work

- Conflict handling is intentionally simple and assumes a primarily single-user, asynchronously used environment.
- Fixed chunks can lose reuse efficiency after insertions or deletions; periodic rebuilding is an MVP trade-off rather than a final chunking strategy.
- SFTP is the only initial transport. A provider abstraction can be added later for object storage or other remote backends.
- Delete propagation semantics need careful testing before treating the implementation as production-safe.
- Sensitive configuration files may need opt-in handling, validation, and machine-specific exclusions.
- Future versions could add stronger conflict preservation, encryption, more transports, richer history, and a background service installer.

14. Positioning

The project is not intended to claim that synchronization, filtering, chunking, or expiring storage are individually new technologies. The value of the concept is the workflow and lifecycle model: synchronization is meant to disappear into normal computer use, while temporary cross-machine state gets an explicit lifetime and a bounded human-recovery mechanism.

Product principle

Permanent data: "This belongs to my working environment; keep it available."

Temporary data: "I need this to follow me for a while; remove the responsibility of remembering cleanup."

Recall: "I missed the deadline; give me one last chance."

Working concept document - features and architecture discussed for the EduRankAI assessment